# BSIT 375: Knowledge Discovery and Data Science

Week 3 Lecture Notes: Bayes Classifier, Decision Trees and Random Forest

Based on Larose & Larose: *Discovering Knowledge in Data (2nd Ed)*

Instructor: Tek Raj Pant

March 2026

# 1 Bayes Classifier

## Conditional Probability and Bayes' Theorem

Before delving into the Bayes classifier, we review two fundamental concepts from probability theory.

**Conditional Probability**

The conditional probability of an event $A$ given that event $B$ has occurred is defined as:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0.$$

This quantifies the likelihood of $A$ under the knowledge that $B$ is true.

**Bayes' Theorem**

Bayes' Theorem provides a way to update the probability of a hypothesis based on observed evidence. For two events $A$ and $B$,

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}.$$

In classification, we treat the class label $C_k$ as the hypothesis and the feature vector $\mathbf{x}$ as the evidence:

$$P(C_k \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid C_k)\, P(C_k)}{P(\mathbf{x})}.$$

The denominator $P(\mathbf{x})$ is a normalizing constant that ensures the posterior probabilities sum to 1.

## The Bayes Decision Rule

The **Bayes classifier** assigns an observation to the class that maximizes the posterior probability $P(C_k \mid \mathbf{x})$. Formally, the decision rule is:

$$\hat{y} = \arg\max_k P(C_k \mid \mathbf{x}) = \arg\max_k \frac{P(\mathbf{x} \mid C_k) P(C_k)}{P(\mathbf{x})}.$$

Since $P(\mathbf{x})$ is constant across classes, the rule simplifies to:

$$\hat{y} = \arg\max_k P(\mathbf{x} \mid C_k) P(C_k).$$

The Bayes classifier minimizes the probability of misclassification (the 0-1 loss) and is therefore the *optimal* classifier if the true class-conditional densities are known.

## Naïve Bayes Classifier

In practice, the joint distribution $P(\mathbf{x} \mid C_k)$ is often unknown and high-dimensional. The **naïve Bayes** classifier makes a strong independence assumption: given the class, all features are conditionally independent. Thus,

$$P(\mathbf{x} \mid C_k) = \prod_{j=1}^{p} P(x_j \mid C_k),$$

where $p$ is the number of features. The decision rule becomes:

$$\hat{y} = \arg\max_k P(C_k) \prod_{j=1}^{p} P(x_j \mid C_k).$$

Despite the unrealistic independence assumption, naïve Bayes often performs surprisingly well in practice, especially for text classification and high-dimensional problems.

### Handling Different Feature Types:

- **Categorical features**: $P(x_j \mid C_k)$ is estimated by the proportion of training instances in class $k$ with that feature value.
- **Continuous features**: Typically assumed to follow a Gaussian distribution:

$$P(x_j \mid C_k) = \frac{1}{\sqrt{2\pi}\sigma_{jk}} \exp\left(-\frac{(x_j - \mu_{jk})^2}{2\sigma_{jk}^2}\right),$$

  where $\mu_{jk}$ and $\sigma_{jk}^2$ are the sample mean and variance of feature $j$ for class $k$.

## Example

Consider the following small dataset for predicting whether a customer will purchase a product based on two features: *Age* (young, middle, old) and *Income* (low, medium, high).

Table 1: Training data for purchase prediction

| ID | Age | Income | Purchase? |
|---|---|---|---|
| 1 | Young | Low | No |
| 2 | Young | Medium | No |
| 3 | Young | High | Yes |
| 4 | Middle | Low | Yes |
| 5 | Middle | Medium | Yes |
| 6 | Middle | High | Yes |
| 7 | Old | Low | No |
| 8 | Old | Medium | No |
| 9 | Old | High | Yes |
| 10 | Old | High | Yes |

We want to classify a new customer with *Age = Young* and *Income = Medium*.

**Step 1: Compute prior probabilities:** Total instances: $n = 10$.

$$P(\text{Yes}) = \frac{6}{10} = 0.6, \quad P(\text{No}) = \frac{4}{10} = 0.4.$$

**Step 2: Estimate class-conditional likelihoods:** For *Purchase = Yes* (6 instances):

- Age: Young (1/6), Middle (3/6 = 0.5), Old (2/6 ≈ 0.3333)
- Income: Low (1/6), Medium (2/6 ≈ 0.3333), High (3/6 = 0.5)

For *Purchase = No* (4 instances):

- Age: Young (2/4 = 0.5), Middle (0/4 = 0), Old (2/4 = 0.5)
- Income: Low (2/4 = 0.5), Medium (2/4 = 0.5), High (0/4 = 0)

**Step 3: Compute posterior probabilities:** For *Yes*:

$$P(\text{Yes} \mid \text{Young, Medium}) \propto P(\text{Yes}) \times P(\text{Young} \mid \text{Yes}) \times P(\text{Medium} \mid \text{Yes})$$

$$= 0.6 \times \frac{1}{6} \times \frac{2}{6} = 0.6 \times 0.1667 \times 0.3333 \approx 0.03333.$$

For *No*:

$$P(\text{No} \mid \text{Young, Medium}) \propto P(\text{No}) \times P(\text{Young} \mid \text{No}) \times P(\text{Medium} \mid \text{No})$$

$$= 0.4 \times 0.5 \times 0.5 = 0.4 \times 0.25 = 0.1.$$

Normalizing: $0.03333 + 0.1 = 0.13333$, so

$$P(\text{Yes} \mid \text{Young, Medium}) = \frac{0.03333}{0.13333} \approx 0.25, \quad P(\text{No} \mid \text{Young, Medium}) = \frac{0.1}{0.13333} \approx 0.75.$$

Since $P(\text{No}) > P(\text{Yes})$, the naïve Bayes classifier predicts **No** for the new customer.

## Advantages and Limitations

**Pros & Cons of Naïve Bayes**

**Advantages:**

- Simple, fast, and scales well to high dimensions.
- Works well with small training data.
- Handles both continuous and discrete features naturally.
- Provides probabilistic predictions.

**Limitations:**

- The independence assumption is often unrealistic, but can still yield good results.
- Poor estimator of probabilities when features are strongly correlated.
- Zero frequency problem: if a feature value never occurs for a class, the product becomes zero. This is mitigated by Laplace smoothing.

## Laplace Smoothing

To avoid zero probabilities when a feature value is missing in the training data for a class, we add a small pseudo-count $\alpha$ (usually 1) to all counts. For a categorical feature with $d$ possible values, the smoothed estimate is:

$$\hat{P}(x_j = v \mid C_k) = \frac{\text{count}(v, C_k) + \alpha}{\text{count}(C_k) + \alpha d}.$$

# 2 Decision Trees

## Introduction

Decision trees are a non-parametric supervised learning method used for classification and regression. They model decisions by recursively partitioning the feature space into regions and assigning a prediction (class label or numeric value) to each region. The resulting structure resembles a tree: internal nodes represent a test on a feature, branches represent outcomes of the test, and leaf nodes contain the final predictions.

**Key Terminology**

- **Root Node**: The topmost node, containing the entire dataset.
- **Internal Nodes**: Nodes that test a feature and split the data.
- **Leaves (Terminal Nodes)**: Nodes that assign a class label or value.
- **Splitting Criterion**: A metric used to choose which feature and threshold to split on (e.g., entropy, Gini).
- **Pruning**: The process of removing parts of the tree to reduce overfitting.

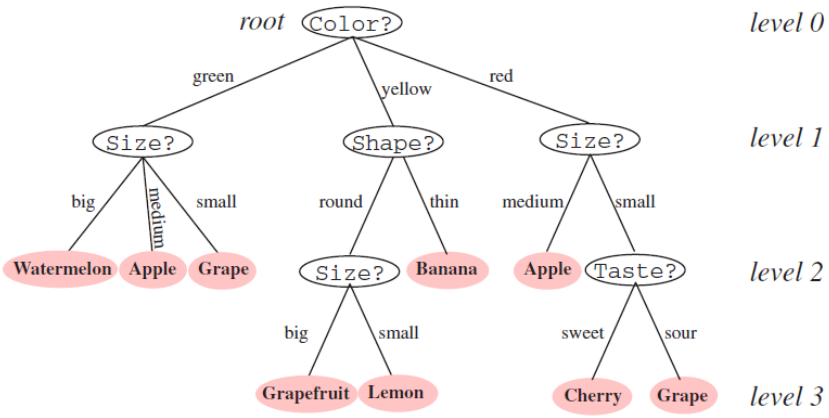


Figure 1: Decision Tree

## Tree Induction Algorithms

Several algorithms exist for building decision trees. The most common are:

- **ID3** (Iterative Dichotomiser 3): Uses information gain (entropy) for categorical features.
- **C4.5**: An extension of ID3 that handles continuous features, missing values, and uses gain ratio.
- **CART** (Classification and Regression Trees): Uses Gini impurity for classification and variance reduction for regression; produces binary trees.

## Splitting Criteria

The choice of splitting criterion determines how a tree is constructed. We focus on two popular measures: **entropy** (used in ID3, C4.5) and **Gini impurity** (used in CART).

**Entropy and Information Gain:** Entropy measures the impurity or uncertainty of a set of data. For a dataset $S$ with $c$ classes, the entropy is defined as:

$$H(S) = -\sum_{i=1}^{c} p_i \log_2 p_i,$$

where $p_i$ is the proportion of instances belonging to class $i$ in $S$. By convention, $0 \log_2 0 = 0$.

If a split partitions $S$ into $k$ subsets $S_1, S_2, \ldots, S_k$ based on a feature $A$, the **information gain** is the reduction in entropy:

$$\text{Gain}(S, A) = H(S) - \sum_{j=1}^{k} \frac{|S_j|}{|S|} H(S_j).$$

The algorithm selects the feature that maximizes the information gain.

**Numerical Example: Information Gain**

Consider a binary classification problem with 10 instances, 6 of class "Yes" and 4 of class "No". The entropy of the root node is:

$$H(S) = -\frac{6}{10}\log_2\frac{6}{10} - \frac{4}{10}\log_2\frac{4}{10} = -0.6\times(-0.73697) - 0.4\times(-1.32193) \approx 0.97095.$$

Suppose we split on feature $A$ that yields two subsets:

- $S_1$: 4 instances (3 Yes, 1 No)
- $S_2$: 6 instances (3 Yes, 3 No)

The entropy after the split is:

$$H_{\text{after}} = \frac{4}{10} H(S_1) + \frac{6}{10} H(S_2).$$

Calculate $H(S_1) = -\frac{3}{4}\log_2\frac{3}{4} - \frac{1}{4}\log_2\frac{1}{4} \approx -0.75(-0.415) - 0.25(-2) \approx 0.81128$, and $H(S_2) = -\frac{3}{6}\log_2\frac{3}{6} - \frac{3}{6}\log_2\frac{3}{6} = -0.5(-1) - 0.5(-1) = 1$. Thus $H_{\text{after}} = 0.4 \times 0.81128 + 0.6 \times 1 = 0.324512 + 0.6 = 0.924512$. Information gain $= 0.97095 - 0.92451 \approx 0.04644$. If another feature yields higher gain, it would be chosen.

**Gini Impurity:** Gini impurity measures the probability of misclassifying a randomly chosen instance if it were randomly labeled according to the class distribution in $S$:

$$\text{Gini}(S) = 1 - \sum_{i=1}^{c} p_i^2.$$

For a binary split, the weighted average Gini after split is:

$$\text{Gini}_{\text{split}} = \sum_{j=1}^{k} \frac{|S_j|}{|S|} \text{Gini}(S_j).$$

The CART algorithm chooses the split that minimizes $\text{Gini}_{\text{split}}$.

**Numerical Example: Gini Impurity**

Using the same dataset ($S$: 6 Yes, 4 No):

$$\text{Gini}(S) = 1 - \left( \left( \frac{6}{10} \right)^2 + \left( \frac{4}{10} \right)^2 \right) = 1 - (0.36 + 0.16) = 0.48.$$

For the split on $A$ described above: $\text{Gini}(S_1) = 1 - \left( (\frac{3}{4})^2 + (\frac{1}{4})^2 \right) = 1 - (0.5625 + 0.0625) = 0.375$, $\text{Gini}(S_2) = 1 - \left( (\frac{3}{6})^2 + (\frac{3}{6})^2 \right) = 1 - (0.25 + 0.25) = 0.5$. Weighted Gini $= \frac{4}{10} \times 0.375 + \frac{6}{10} \times 0.5 = 0.15 + 0.3 = 0.45$, which is lower than the original 0.48, indicating improvement.

## Handling Continuous Features

In C4.5 and CART, continuous features are handled by considering binary splits based on a threshold $t$. The algorithm sorts the values of the feature, then evaluates potential thresholds between consecutive distinct values, choosing the one that maximizes information gain (or minimizes Gini).

## Overfitting and Pruning

Decision trees can grow too deep, capturing noise in the training data—this is overfitting. Two strategies prevent overfitting:

- **Pre-pruning (early stopping)**: Stop growing the tree when splitting no longer provides a significant gain, or when a node contains too few instances.
- **Post-pruning**: Grow the full tree and then remove branches that have little contribution. CART uses *cost-complexity pruning*, where a complexity parameter $\alpha$ penalizes the number of leaves:

  $$R_\alpha(T) = R(T) + \alpha |\tilde{T}|,$$

  where $R(T)$ is the misclassification rate on training data and $|\tilde{T}|$ is the number of leaves.

## Example

Consider the following small dataset for a "Play Tennis" decision (based on Quinlan's classic example):

Table 2: Weather dataset

| Outlook | Temperature | Humidity | Windy | Play? |
|---|---|---|---|---|
| Sunny | Hot | High | False | No |
| Sunny | Hot | High | True | No |
| Overcast | Hot | High | False | Yes |
| Rainy | Mild | High | False | Yes |
| Rainy | Cool | Normal | False | Yes |
| Rainy | Cool | Normal | True | No |
| Overcast | Cool | Normal | True | Yes |
| Sunny | Mild | High | False | No |
| Sunny | Cool | Normal | False | Yes |
| Rainy | Mild | Normal | False | Yes |
| Sunny | Mild | Normal | True | Yes |
| Overcast | Mild | High | True | Yes |
| Overcast | Hot | Normal | False | Yes |
| Rainy | Mild | High | True | No |

We will build a decision tree using information gain. First, compute entropy of the target: Total instances: 14, Yes = 9, No = 5.

$$H(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} \approx -0.6429(-0.637) - 0.3571(-1.485) \approx 0.940.$$

Now evaluate candidate splits. For Outlook (three values):

- Sunny: 5 instances (2 Yes, 3 No) $\rightarrow H = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} \approx 0.9710$.
- Overcast: 4 instances (4 Yes, 0 No) $\rightarrow H = 0$.
- Rainy: 5 instances (3 Yes, 2 No) $\rightarrow H = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} \approx 0.9710$.

Weighted entropy after split:

$$H_{\text{after}} = \frac{5}{14} \times 0.9710 + \frac{4}{14} \times 0 + \frac{5}{14} \times 0.9710 = 0.6936.$$

Information gain $= 0.940 - 0.6936 = 0.2464$.

For Humidity (binary):

- High: 7 instances (3 Yes, 4 No) $\rightarrow H = -\frac{3}{7}\log_2\frac{3}{7} - \frac{4}{7}\log_2\frac{4}{7} \approx 0.9852$.
- Normal: 7 instances (6 Yes, 1 No) $\rightarrow H = -\frac{6}{7}\log_2\frac{6}{7} - \frac{1}{7}\log_2\frac{1}{7} \approx 0.5917$.

Weighted entropy $= \frac{7}{14}(0.9852) + \frac{7}{14}(0.5917) = 0.78845$. Gain $= 0.940 - 0.78845 = 0.15155$.

For Windy:

- True: 6 instances (3 Yes, 3 No) $\rightarrow H = 1$.
- False: 8 instances (6 Yes, 2 No) $\rightarrow H = -\frac{6}{8}\log_2\frac{6}{8} - \frac{2}{8}\log_2\frac{2}{8} \approx 0.8113$.

Weighted $= \frac{6}{14}(1) + \frac{8}{14}(0.8113) = 0.8922$, gain $= 0.0478$.

For Temperature (three values):

- Hot: 4 instances (2 Yes, 2 No) $\rightarrow H = 1$.
- Mild: 6 instances (4 Yes, 2 No) $\rightarrow H = -\frac{4}{6}\log_2\frac{4}{6} - \frac{2}{6}\log_2\frac{2}{6} \approx 0.9183$.
- Cool: 4 instances (3 Yes, 1 No) $\rightarrow H = -\frac{3}{4}\log_2\frac{3}{4} - \frac{1}{4}\log_2\frac{1}{4} \approx 0.8113$.

Weighted $= \frac{4}{14}(1) + \frac{6}{14}(0.9183) + \frac{4}{14}(0.8113) = 0.2857 + 0.3936 + 0.2318 = 0.9111$, gain $= 0.0289$.

Outlook gives the highest gain, so it becomes the root. The subtree for Sunny is then split further, and the process repeats. The final tree can be drawn with leaves representing decisions.

### Advantages and Disadvantages

**Pros & Cons**

**Advantages:**

- Easy to interpret and visualize.
- No feature scaling required.
- Can handle both categorical and numerical data.
- Mimics human decision-making.

**Disadvantages:**

- Prone to overfitting without pruning.
- Instability: small changes in data can lead to a completely different tree.
- Biased towards features with many levels (if not using gain ratio).

# 3 Random Forest

## Ensemble Learning

An ensemble combines multiple models to produce a single, more accurate prediction. Random Forest is an ensemble of decision trees, each built on a different bootstrap sample of the data and with a random subset of features considered at each split.

## Bagging (Bootstrap Aggregating)

Bagging reduces variance by averaging predictions from models trained on bootstrapped datasets. Given original data $D$ of size $n$, we create $B$ bootstrap samples $D_1, \ldots, D_B$ by sampling $n$ observations with replacement. For each $D_b$, we train a tree $T_b$. For classification, the final prediction is the majority vote:

$$\hat{y} = \text{mode}\{T_1(x), T_2(x), \ldots, T_B(x)\}.$$

For regression, it is the average.

Theoretical insight: If the models are independent, the variance of the average is $\sigma^2/B$, where $\sigma^2$ is the variance of an individual model. However, bootstrapped trees are correlated, so the reduction is less but still significant.

## Random Forest Algorithm

Random Forest adds an extra layer of randomness: at each node, instead of considering all features, a random subset of $m$ features is chosen (typically $m = \sqrt{p}$ for classification, $p/3$ for regression). The best split among those $m$ features is used. This decorrelates the trees, further reducing variance.

**Algorithm 1** Random Forest

**Input**: Training data $D$, number of trees $B$, number of features to try $m$.
**for** $b = 1$ to $B$ **do**
    Bootstrap sample $D_b$ from $D$.
    Grow a tree $T_b$ from $D_b$:
    **while** stopping criterion not met **do**
        Randomly select $m$ features from all features.
        Find the best split among those $m$ features (using Gini or entropy).
        Split node into child nodes.
    **end while**
**end for**
**Return** ensemble $\{T_b\}_{b=1}^{B}$.

### Out-of-Bag (OOB) Error

For each tree, about $1/e \approx 36.8\%$ of the original data is not included in its bootstrap sample (out-of-bag). These OOB instances can serve as a validation set. The OOB error is computed by aggregating predictions for each instance from the trees that did not use it, providing an unbiased estimate of generalization error without needing cross-validation.

### Variable Importance

Random Forest provides two measures of feature importance:

1. **Mean Decrease Impurity (MDI)**: Sum of decreases in impurity (Gini or entropy) over all splits where the feature is used, averaged across trees.
2. **Permutation Importance**: For each feature, randomly shuffle its values and compute the increase in OOB error. A larger increase indicates higher importance.

### Numerical Illustration

Consider a small dataset with 5 features and 20 instances, binary classification. We build $B = 3$ trees with $m = 2$ (since $\sqrt{5} \approx 2.2$). The OOB error is computed as follows:

- For tree 1: bootstrap sample includes instances 1,3,5,7,9,11,13,15,17,19,1,3 (some repeated). OOB set = 2,4,6,8,10,12,14,16,18,20.
- For tree 2: bootstrap includes a different set, and so on.

For instance 1, suppose it is OOB for trees 2 and 3, and those trees predict class 0 and 1 respectively. The majority is 1. Compare with true label to compute OOB error. After all instances, the OOB error rate is the fraction of misclassifications.

### Hyperparameters

Key parameters in Random Forest:

- `n_estimators` ($B$): Number of trees. More trees improve performance but increase computational cost.
- `max_features` ($m$): Number of features to consider at each split.
- `min_samples_leaf`: Minimum number of samples required to be at a leaf node (controls tree depth).
- `bootstrap`: Whether to use bootstrap samples (default True).

### Advantages and Limitations

**Random Forest: Pros & Cons**

**Advantages:**

- High accuracy, robust to overfitting.
- Provides feature importance.
- Handles high dimensionality and missing data well.
- OOB error eliminates need for separate validation.

**Limitations:**

- Less interpretable than a single decision tree.
- Can be memory-intensive (stores all trees).
- May be biased towards categorical features with many levels (though less than a single tree).

## 4 Comparison and Summary

Decision trees are interpretable but prone to overfitting. Random forests improve predictive accuracy and robustness by averaging many trees, sacrificing some interpretability for performance. In practice, random forests are a go-to method for many classification and regression tasks due to their excellent out-of-the-box performance.

### When to Use Which?

- Use a single decision tree when interpretability is paramount and the data is simple.
- Use random forest when prediction accuracy is the priority and interpretability can be traded off (though feature importance still offers insights).

### Bayes Classifier, Decision Trees and Random Forest

While naïve Bayes assumes feature independence, decision trees and random forests capture interactions between features through recursive partitioning. In practice, random forests often outperform naïve Bayes when interactions are important, but naïve Bayes remains a strong baseline, especially for text data where independence approximations are reasonable.

## Further Reading


- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- Quinlan, J. R. (1993). *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
- Larose, D. T., & Larose, C. D. (2014). *Discovering Knowledge in Data: An Introduction to Data Mining*. John Wiley & Sons.